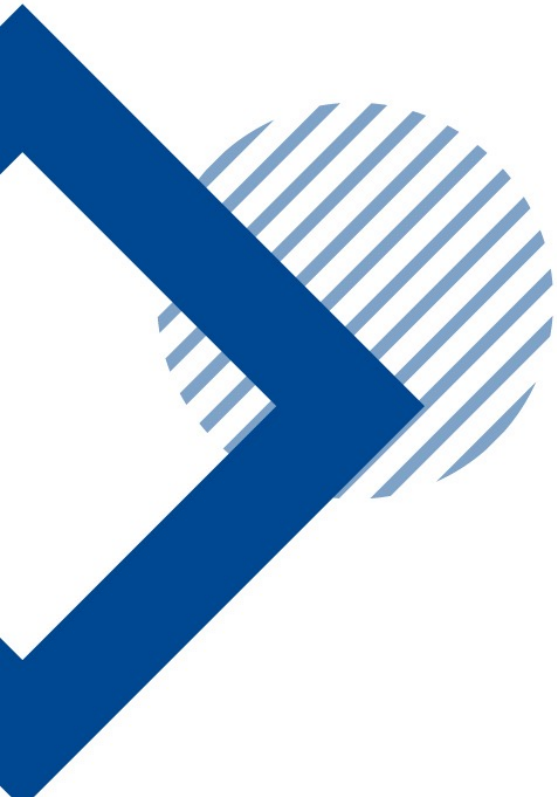
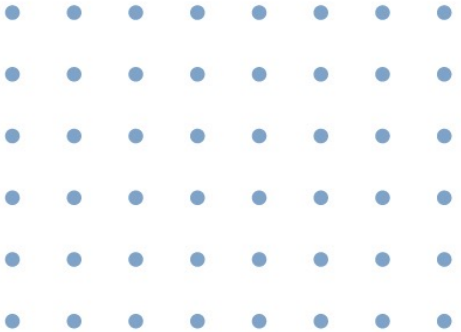
SPA, React

Интернет-программирование







2022

Одностраничное приложение - 1

Одностраничное приложение (single page application, SPA) — это веб-приложение или веб-сайт, использующий единственный HTML-документ как оболочку для всех веб-страниц и организующий взаимодействие с пользователем через динамически подгружаемые HTML, CSS, JavaScript, обычно посредством AJAX.

Одностраничные приложения напоминают родные (native) приложения, с той лишь разницей, что исполняются в рамках браузера, а не в собственном процессе операционной системы.

Одностраничное приложение - 2

Основными элементами, использующимися при построении SPA, являются:

- JavaScript-фреймворки
- Роутинг: навигация между представлениями производится во фронтенде
- Шаблонизатор
- HTML5
- API для бэкенда, например, в стиле REST
- AJAX

React

React (иногда React.js или ReactJS) — JavaScript-библиотека с открытым исходным кодом для разработки пользовательских интерфейсов.

Реактивное программирование — парадигма программирования, ориентированная на потоки данных и распространение изменений.
Современные табличные процессоры представляют собой пример реактивного программирования. Ячейки таблицы могут содержать строковые значения или формулу вида «=B1+C1», значение которой будет вычислено исходя из значений соответствующих ячеек. Когда значение одной из зависимых ячеек будет изменено, значение этой ячейки будет автоматически обновлено.

JavaScript XML (JSX) — это расширение синтаксиса JavaScript, которое позволяет использовать HTML-подобный синтаксис для описания структуры интерфейса. Как правило, компоненты написаны с использованием JSX, но также есть возможность использования обычного JavaScript.

React – JSX 1

Объявление переменной

```
const element = <h1>Привет, мир!</h1>;
```

Шаблонизатор

```
const name = 'Иван-Царевич';
const element = <h1>Здравствуй, {name}!</h1>;
```

Вызов функций

```
function formatName(user) {
  return user.firstName + ' ' + user.lastName;
}

const user = {
  firstName: 'Марья',
  lastName: 'Моревна'
};

const element = (
  <h1>
    Здравствуй, {formatName(user)}!
  </h1>
);
```

React – JSX 2

Атрибуты

```
const element = <a href="https://www.reactjs.org"> link </a>;
```

```
const element = <img src={user.avatarUrl}></img>;
```

Дочерние элементы

```
const element = (
  <div>
    <h1>Здравствуйте!</h1>
    <h2>Рады вас видеть.</h2>
  </div>
);
```

React – Компоненты и свойства

Объявление компонента

```
function Welcome({name}) {
  return <h1>Привет, {name}</h1>;
}
```

Свойства компонента доступны только для чтения!

```
const root = ReactDOM.createRoot(document.getElementById('root'));
const element = <Welcome name="Алиса" />;
root.render(element);
```

Композиция компонентов

```
function Welcome({name}) {
  return <h1>Привет, {name}</h1>;
}

function App() {
  return (
    <div>
      <Welcome name="Алиса" />
      <Welcome name="Базилио" />
      <Welcome name="Буратино" />
    </div>
  );
}
```

React – Состояния

Использование состояний

```
function Groups() {
    const [groups, setGroups] = useState([]);
    useEffect(() => {
        Ajax.getGroups().then(data => setGroups(data));
    }, []);
}
```

- *Жизненный цикл componentDidMount*

```
useEffect(() => {
  // Your code here
}, []);
```

- *Жизненный цикл componentDidUpdate*

```
useEffect(() => {
  // Your code here
}, [yourDependency]);
```

- *Жизненный цикл componentWillUnmount*

```
useEffect(() => {
return () => {
    // Your code here
  }
}, [yourDependency]);
```

React – Обработка событий

Обработка события нажатия на кнопку

```
<button onClick={activateLasers}>
  Активировать лазеры
</button>
```

Отключение стандартной обработки

```
function Form() {
  function handleSubmit(e) {
    e.preventDefault();
    console.log('Отправлена форма.');
  }

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Отправить</button>
    </form>
  );
}
```

Передача аргументов

```
<button onClick={(e) => deleteRow(id, e)}>Удалить строку</button>
```

React – Условный рендеринг 1

```
function UserGreeting() {
  return <h1>С возвращением!</h1>;
}

function GuestGreeting() {
  return <h1>Войдите, пожалуйста.</h1>;
}

function Greeting({isLoggedIn}) {
  if (isLoggedIn) {
    return <UserGreeting />;
  }
  return <GuestGreeting />;
}
```

React – Условный рендеринг 2

```
let button;
if (isLoggedIn) {
  button = <LogoutButton onClick={handleLogoutClick} />;
} else {
  button = <LoginButton onClick={handleLoginClick} />;
}

return (
  <div>
    <Greeting isLoggedIn={isLoggedIn} />
    {button}
  </div>
);
```

React – Условный рендеринг 3

```
function Mailbox({unreadMessages}) {
  return (
   <div>
    <h1>Здравствуйте!</h1>
    {unreadMessages.length > 0 &&
     <h2>
      У вас {unreadMessages.length} непрочитанных сообщений.
     </h2>
    }
   </div>
 );
}
```

Приведённый выше вариант работает корректно, потому что в JavaScript-выражение true && expression всегда вычисляется как expression, а выражение false && expression — как false.

React – Условный рендеринг 4

```
render() {
  return (
   <div>
     Пользователь <b>{isLoggedIn ? 'сейчас' : 'не'}</b> на сайте.
   </div>
 );
}

render() {
  return (
   <div>
     {isLoggedIn
       ? <LogoutButton onClick={handleLogoutClick} />
       : <LoginButton onClick={handleLoginClick} />
     }
   </div>
 );
}
```

React – Условный рендеринг 5

Предотвращение рендеринга компонента

```
function WarningBanner({warn}) {
  if (warn) {
    return null;
  }

  return (
    <div className="warning">
      Предупреждение!
    </div>
  );
}
```

В редких случаях может потребоваться позволить компоненту спрятать себя, хотя он уже и отрендерен другим компонентом. Чтобы этого добиться, верните null вместо того, что обычно возвращается на рендеринг.

React – Списки и ключи 1

```
function NumberList({numbers}) {
  const listItems = numbers.map((number) =>
   <li>{number}</li>
 );
 return (
   <ul>{listItems}</ul>
 );
}

const numbers = [1, 2, 3, 4, 5];
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<NumberList numbers={numbers} />);
```

React – Списки и ключи 2

```
function NumberList({numbers}) {
  const listItems = numbers.map((number) =>
   <li key={number.toString()}>
    {number}
   </li>);
 return (
  <ul>{listItems}</ul>
 );
}
```

Ключи помогают React определять, какие элементы были изменены, добавлены или удалены. Ключи должны быть уникальными среди соседних элементов (в одном цикле).

```
const todoItems = todos.map((todo) =>
 <li key={todo.id}>
  {todo.text}
 </li>
);
const todoItems = todos.map((todo, index) =>
 // Делайте так, только если у элементов массива нет заданного ID
 <li key={index}>
  {todo.text}
 </li>
);
```

React – Списки и ключи 3

Извлечение компонентов с ключами

```
function ListItem({value}) {
  // Правильно! Не нужно определять здесь ключ:
  return <li>{value}</li>;
}

function NumberList({numbers}) {
    const listItems = numbers.map((number) =>
    // Правильно! Ключ нужно определять внутри массива:
    <ListItem key={number.toString()} value={number} />
  );
  return (
    <ul>
      {listItems}
    </ul>
  );
}
```

React – Списки и ключи 4

Встраивание map() в JSX

```
function NumberList({numbers}) {
  return (
   <ul>
    {
      numbers.map((number) =>
      <ListItem key={number.toString()}
              value={number} />)
   }
   </ul>
 );
}
```

Спасибо за внимание!

Литература:

1. https://react.dev/learn – Документация по React
2. https://react.dev/learn/adding-interactivity – Добавление интерактивности
3. https://react.dev/learn/managing-state – Управление состоянием компонентов
4. https://react-bootstrap.netlify.app/docs/components/accordion – React bootstrap



2022